

深度学习归一化

导论

深度神经网络由很多层连续组成。前一层输出会直接成为后一层输入，所以只要中间表示的数值尺度不断变化，后面所有层的计算都会跟着改变。假设某一层原本输出大约在 $[-1, 1]$ ，训练若干步以后逐渐变成 $[-20, 20]$ ，那么同样一组后续权重会收到完全不同量级的输入，激活函数、Attention Logit、梯度和学习率都会受到影响。归一化要解决的主要问题，就是让网络内部的表示始终处于比较稳定、容易优化的数值范围。

最早的做法主要是对输入数据进行标准化。2015 年 Batch Normalization (BN) 把这种思想放进了网络内部，并在 CNN 时代获得巨大成功。BN 会对同一个 mini-batch 中的样本一起统计均值和方差，因此它很依赖 batch。2016 年 Layer Normalization (LN) 把统计范围改到单个样本内部，从而摆脱 batch size 的影响；InstanceNorm 和 GroupNorm 又把统计范围进一步细分到单张图片、单个通道或通道组。Transformer 出现以后，LayerNorm 因为每个 Token 可以独立计算，很快成为序列模型常用的归一化方式。

2019 年 RMSNorm 继续简化 LayerNorm。LayerNorm 会先减去均值，再除以标准差；RMSNorm 直接根据平方平均控制向量尺度，保留缩放归一化，省去均值中心化。这个变化很小，但非常适合深层 Transformer：它和 batch 无关，训练与推理公式一致，可以自然放进 Pre-Norm 残差结构，同时计算步骤更少。LLaMA、Mistral、Qwen、Gemma 等公开大模型都采用过 RMSNorm 或与其等价的结构。

这节课沿着“输入标准化 $\rightarrow$ BatchNorm $\rightarrow$ LayerNorm $\rightarrow$ Pre-Norm $\rightarrow$ RMSNorm”这条主线展开，同时穿插 WeightNorm、InstanceNorm 和 GroupNorm。重点放在三个问题：统计量到底沿哪些维度计算，归一化究竟改变了向量的什么性质，以及这些变化怎样一步一步适应更深、更大的 Transformer。

BatchNorm: <https://proceedings.mlr.press/v37/ioffe15.html>

WeightNorm: <https://arxiv.org/abs/1602.07868>

LayerNorm: <https://arxiv.org/abs/1607.06450>

InstanceNorm: <https://arxiv.org/abs/1607.08022>

GroupNorm: https://openaccess.thecvf.com/content_ECCV_2018/html/Yuxin_Wu_Group_Normalization_ECCV_2018_paper.html

RMSNorm: <https://proceedings.neurips.cc/paper/2019/hash/1e8a19426224ca89e83cef47f1e7f53b-Abstract.html>

一、标准化

先从一个普通向量开始。设

$$x = (x_1, x_2, \dots, x_d).$$

如果不同维度的量级差别很大，例如某些维度大约是 0.1，另一些维度大约是 100，那么后续线性层 $y = Wx + b$ 中，数值较大的维度会更容易主导输出。训练过程中只要前面几层的参数稍有变化，这种尺度差异还可能继续放大。

最常见的标准化先计算均值

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i,$$

再计算方差

$$\sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2,$$

最后得到

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}.$$

这里 ϵ 是很小的正数，用于避免方差接近 0 时出现数值问题。

这个公式包含两步。 $x_i - \mu$ 负责把所有维度整体平移到均值附近，叫做中心化；除以 $\sqrt{\sigma^2 + \epsilon}$ 负责把整体波动大小压到统一尺度，叫做尺度归一化。忽略 ϵ 后，标准化结果的均值为 0，方差为 1。

假设 $x = [2, 4, 6, 8]$ 。均值是 $\mu = 5$ ，所以中心化以后得到 $[-3, -1, 1, 3]$ 。四个数平方以后是 $[9, 1, 1, 9]$ ，平均值为 5，因此标准差约为 $\sqrt{5} \approx 2.236$ 。最终标准化结果约为 $[-1.34, -0.45, 0.45, 1.34]$ 。

如果原始向量整体乘 10，变成 $[20, 40, 60, 80]$ ，均值和标准差也都会乘 10，所以标准化结果仍然几乎一样。这个例子说明，标准化会显著减弱“整体数值变大或变小”对后续网络的影响。

神经网络里的归一化通常还会加入可学习参数

$$y_i = \gamma_i \hat{x}_i + \beta_i.$$

γ_i 负责重新调整第 i 个维度的幅值， β_i 负责重新调整偏移。归一化先把输入整理到稳定尺度，再让模型自己学习真正需要的尺度和偏移，所以它不会限制网络的表达能力。

二、BatchNorm



图 1 标准化由中心化、尺度归一化和可学习仿射变换组成。

Batch Normalization 由 Ioffe 和 Szegedy 在 2015 年提出。论文: *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift* (<https://proceedings.mlr.press/v37/ioffe15.html>)。

假设某一层输出

$$X \in \mathbb{R}^{B \times D},$$

其中 B 是 batch size, D 是特征维度。BN 的做法是: 固定某一个特征 j , 把 batch 中所有样本的第 j 个值拿出来, 一起计算均值和方差:

$$\mu_j = \frac{1}{B} \sum_{b=1}^B X_{bj}, \quad \sigma_j^2 = \frac{1}{B} \sum_{b=1}^B (X_{bj} - \mu_j)^2.$$

然后得到

$$\hat{X}_{bj} = \frac{X_{bj} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}, \quad Y_{bj} = \gamma_j \hat{X}_{bj} + \beta_j.$$

BN 最重要的特点可以直接从统计方向看出来: 同一个样本内部的不同特征不会一起统计, 同一个特征会沿 batch 方向收集多个样本。假设 $B = 4, D = 3$, 输入是

$$X = \begin{bmatrix} 1 & 10 & 100 \\ 2 & 20 & 90 \\ 3 & 30 & 110 \\ 4 & 40 & 100 \end{bmatrix}.$$

BN 会分别处理三列。第一列用 $[1, 2, 3, 4]$ 计算统计量, 第二列用 $[10, 20, 30, 40]$, 第三列用 $[100, 90, 110, 100]$ 。因此最终得到 3 组均值和 3 组方差。

只看第一列 $[1, 2, 3, 4]$ 。均值是 2.5, 方差是 1.25, 所以归一化以后大约得到 $[-1.34, -0.45, 0.45, 1.34]$ 。这里每一个样本的结果都依赖另外三个样本。如果把第四个样本从 4 改成 40, 其他三个样本的归一化结果也会一起改变, 因为新的 batch 均值和方差都变了。

CNN 中的输入一般写成 $X \in \mathbb{R}^{B \times C \times H \times W}$ 。BN 通常固定通道 c , 沿 B, H, W 三个维度共同统计。也就是说, 一个通道只得到一组均值和方差。卷积通道往往代表相同类型的局部特征, 所以让不同图片、不同空间位置共享同一组通道统计量非常自然。

原始 BN 论文主要用 Internal Covariate Shift 解释它的作用: 前面网络不断更新时, 后面层看到的输入分布也在持续变化, BN 希望减轻这种变化。后来的研究

发现，BN 的收益还和优化平滑性、激活尺度控制、较大学习率下的稳定性有关，因此今天更适合把 BN 理解为一种持续约束中间激活尺度的重新参数化方式。

三、BatchNorm 的限制

BN 在 CNN 中非常有效，但它对 batch 的依赖会带来几个现实问题。第一个问题是小 batch。假设 batch size 为 64，每个特征用 64 个样本估计均值和方差，统计量通常比较稳定；如果 batch size 只有 2，均值和方差会被两个样本强烈影响，波动明显增大。

假设某个特征在 batch size 为 2 时只有两个数 $[1, 9]$ ，均值是 5。如果下一批恰好变成 $[4, 6]$ ，均值仍然是 5，但方差从 16 变成 1。即使两批数据的中心位置完全相同，BN 的缩放强度也会差很多。小 batch 下这种统计波动会直接进入网络。

第二个问题是训练和推理计算不同。训练阶段使用当前 mini-batch 的统计量，推理阶段通常使用训练期间积累的 running mean 和 running variance：

$$\text{train: } (\mu_B, \sigma_B^2), \quad \text{eval: } (\mu_{\text{run}}, \sigma_{\text{run}}^2).$$

因此 BN 需要维护额外状态，部署时还要正确切换 train/eval 模式。

第三个问题出现在序列模型。文本序列长度会变化，真实推理常常一次只处理一个请求或少量请求。让某个 Token 的归一化结果依赖同一 batch 中另外几句话，会让统计逻辑变得很别扭。归一化因此开始转向单样本内部。

四、WeightNorm

Weight Normalization 由 Salimans 和 Kingma 在 2016 年提出。论文： *Weight Normalization: A Simple Reparameterization to Accelerate Training of Deep Neural Networks* (<https://arxiv.org/abs/1602.07868>)。

WeightNorm 处理的是权重。设一个神经元的权重向量为 w ，重新写成

$$w = g \frac{v}{\|v\|_2}.$$

其中 v 决定方向， g 决定整体长度。这样优化器调整“往哪个方向看”和“权重整体多大”时，可以使用相对独立的参数。

设 $v = [3, 4]$ ，则 $\|v\|_2 = 5$ 。如果 $g = 10$ ，那么 $w = 10[3/5, 4/5] = [6, 8]$ 。当优化器只改变 g 时，权重方向保持不变；只改变 v 的方向时，也不会直接要求模型同步处理一个完全不同的整体长度。

WeightNorm 不使用 batch 统计，所以不会出现 BN 的 batch size 问题。不过它控制的是参数本身，无法直接保证实际激活 Wx 的分布始终稳定。它展示了归一化的另一条路线：通过重新参数化把尺度和方向拆开。

五、LayerNorm

Layer Normalization 由 Ba、Kiros 和 Hinton 在 2016 年提出。论文： *Layer Normalization* (<https://arxiv.org/abs/1607.06450>)。

LayerNorm 把统计范围放回单个样本内部。假设 Transformer 中一个 Token 的隐藏向量为

$$x = (x_1, \dots, x_d) \in \mathbb{R}^d.$$

LN 直接在这 d 个隐藏维度上计算

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i, \quad \sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2,$$

然后

$$\text{LN}(x)_i = \gamma_i \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_i.$$

假设 Transformer 隐藏状态为 $X \in \mathbb{R}^{B \times T \times d}$ 。LN 会固定 batch 位置 b 和 Token 位置 t ，只沿最后一个 hidden 维度 d 统计。因此总共会有 $B \times T$ 组独立均值和方差。

设一个 Token 的 hidden vector 是 $x = [1, 2, 3, 4]$ 。均值是 2.5，中心化得到 $[-1.5, -0.5, 0.5, 1.5]$ ，方差为 1.25。归一化后大约是 $[-1.34, -0.45, 0.45, 1.34]$ 。

同一个 batch 中另一个 Token 即使是 $[100, 101, 102, 103]$ ，它也会自己计算均值 101.5 和方差 1.25，归一化后得到几乎相同的相对结构。两个 Token 之间不会共享统计量。

这就是 LN 很适合序列模型的原因。batch size 从 64 变成 1，某个 Token 的计算完全不变；训练和推理也使用同一个公式，不需要 running mean 和 running variance。

LayerNorm 还具有平移和缩放方面的重要性质。若所有维度同时加一个常数 b ，得到 $x' = x + b\mathbf{1}$ ，新的均值变成 $\mu' = \mu + b$ ，所以 $x' - \mu'\mathbf{1} = x - \mu\mathbf{1}$ 。整体平移会被中心化消掉。若所有维度同时乘一个正数 a ，标准差也会乘 a ，最终归一化结果仍然基本保持不变。

六、InstanceNorm 与 GroupNorm

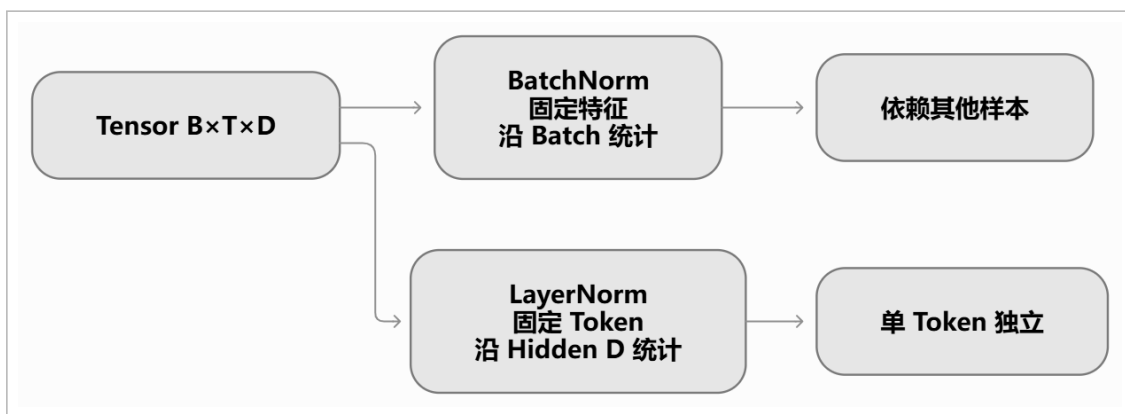


图 2 BatchNorm 与 LayerNorm 的统计方向不同。

Instance Normalization 由 Ulyanov 等人在 2016 年提出。论文: *Instance Normalization: The Missing Ingredient for Fast Stylization* (<https://arxiv.org/abs/1607.08022>)。对于 $X \in \mathbb{R}^{B \times C \times H \times W}$, IN 固定一个样本 b 和一个通道 c , 只在 $H \times W$ 空间上统计。也就是说, 每张图片的每个通道都有自己的均值和方差。

假设一张图片只有 2 个通道, 每个通道是 2×2 。第一个通道为

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix},$$

第二个通道为

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix}.$$

IN 会分别对这两个 2×2 通道计算统计量。第二个通道虽然数值整体大 10 倍, 归一化以后仍会落到和第一个通道相近的标准尺度。

Group Normalization 由 Wu 和 He 在 2018 年提出。论文: *Group Normalization* (https://openaccess.thecvf.com/content_ECCV_2018/html/Yuxin_Wu_Group_Normalization_ECCV_2018_paper.html)。GN 把通道分成若干组, 然后在每个样本的每一组内部, 把组内通道和空间位置一起统计。

假设 $C = 8$, 选择 $G = 4$, 那么每组 2 个通道。第 1、2 通道一起计算一组均值和方差, 第 3、4 通道一起计算另一组, 以此类推。当 $G = 1$ 时, 所有通道在同一组, 行为接近视觉上的 LayerNorm; 当 $G = C$ 时, 每组只有一个通道, 行为接近 InstanceNorm。

BN、LN、IN、GN 可以统一理解为“公式相似，统计集合不同”。对任意统计集合 S ，先算

$$\mu_S = \frac{1}{|S|} \sum_{i \in S} x_i, \quad \sigma_S^2 = \frac{1}{|S|} \sum_{i \in S} (x_i - \mu_S)^2,$$

再做标准化。BN 让 S 跨样本，LN 让 S 位于单个样本内部，IN 把 S 收缩到单样本单通道，GN 则位于 LN 和 IN 之间。

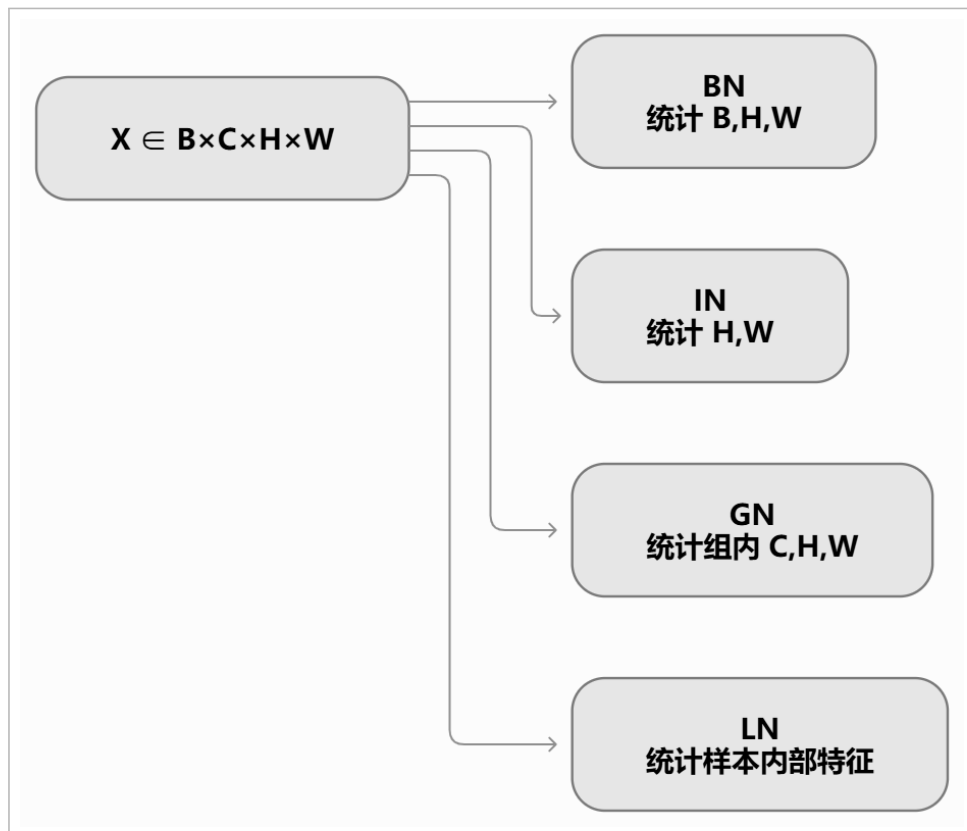


图 3 BN、LN、IN、GN 的主要差异来自统计轴。

七、Transformer

Transformer 在 2017 年提出后，LayerNorm 开始成为序列模型中的标准组件。
原论文： *Attention Is All You Need* (<https://arxiv.org/abs/1706.03762>)。

原始 Transformer 常写成 Post-LN：

$$y = \text{LN}(x + F(x)),$$

其中 F 可以是 Multi-Head Attention，也可以是 FFN。先计算子层输出，与残差相加，再经过 LayerNorm。

假设某层输入为 x ，Attention 输出为 $F(x)$ 。如果两者直接相加，残差流的尺度会随着层数不断变化；Post-LN 会在每一层结尾把结果重新拉回稳定尺度。这个结构在中等深度 Transformer 中工作得很好。

随着网络继续加深，研究者开始关注梯度需要连续穿过很多 LayerNorm 的问题，于是归一化位置逐渐发生变化。

八、Pre-Norm

Pre-Norm 把归一化移动到子层输入：

$$y = x + F(\text{LN}(x)).$$

2020 年 Xiong 等人的论文 *On Layer Normalization in the Transformer Architecture* (<https://proceedings.mlr.press/v119/xiong20b.html>) 系统分析了 Post-LN 和 Pre-LN 的初始化梯度行为。

Pre-Norm 的重要点可以从残差展开看出来。第 l 层写成

$$x_{l+1} = x_l + F_l(\text{LN}(x_l)).$$

连续展开以后

$$x_L = x_0 + \sum_{l=0}^{L-1} F_l(\text{LN}(x_l)).$$

最前面的 x_0 可以沿着残差主路径直接传到深层，中间不会被每一个 LayerNorm 改写。

假设网络只有 3 层。Pre-Norm 可以写成

$$x_1 = x_0 + F_0(\text{LN}(x_0)),$$

$$x_2 = x_1 + F_1(\text{LN}(x_1)),$$

$$x_3 = x_2 + F_2(\text{LN}(x_2)).$$

把前两式继续代入第三式，可以看到 x_3 里面始终保留一条直接的 x_0 项。深层网络中的信息和梯度因此拥有更直接的残差通路。

现代大模型即使普遍仍会配合 warmup、合适初始化和优化器技巧，Pre-Norm 依然成为 Decoder Transformer 中非常常见的设计。接下来真正需要考虑的是：放在 Pre-Norm 位置上的归一化算子能否继续简化。

九、RMSNorm

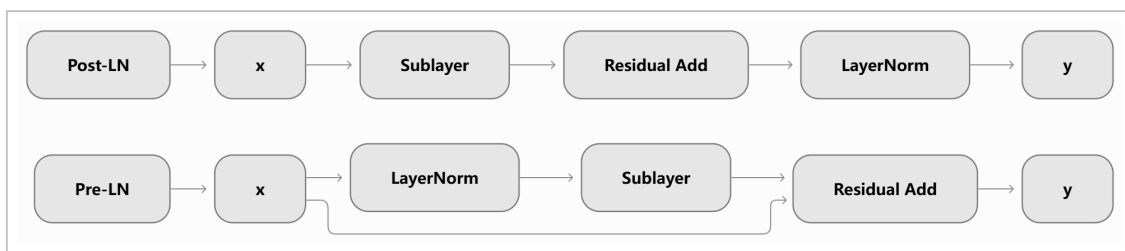


图 4 Post-LN 与 Pre-LN 的归一化位置不同。

RMSNorm 由 Zhang 和 Sennrich 在 2019 年提出。论文: *Root Mean Square Layer Normalization*(<https://proceedings.neurips.cc/paper/2019/hash/1e8a19426224ca89e83cef47.html>)。

先把 LayerNorm 再写一遍:

$$\text{LN}(x)_i = \gamma_i \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_i.$$

它包含两个统计动作: 先减均值完成中心化, 再除标准差控制尺度。

RMSNorm 删除中心化, 只计算均方根

$$r = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon},$$

然后

$$\text{RMSNorm}(x)_i = \gamma_i \frac{x_i}{r}.$$

所以 RMSNorm 只保留“控制向量整体大小”这一部分。它通常没有 β , 也不会计算 μ 。理解这一变化最重要的公式, 是 RMS 和方差之间的关系:

$$\sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2 = \frac{1}{d} \sum_{i=1}^d x_i^2 - \mu^2.$$

因此

$$\text{RMS}^2(x) = \frac{1}{d} \sum_{i=1}^d x_i^2 = \sigma^2 + \mu^2.$$

LayerNorm 根据“去掉均值以后还剩多少波动”缩放; RMSNorm 根据“整个向量的平方平均有多大”缩放。两者都能控制尺度, 只是 RMSNorm 不把整体均值方向删除。

设 $x = [2, -1, 3, 0]$ 。均值为 1。LayerNorm 会先变成 $[1, -2, 2, -1]$, 再按标准差缩放。

RMSNorm 直接计算平方平均: $(4+1+9+0)/4 = 3.5$, 所以 $r = \sqrt{3.5} \approx 1.87$ 。归一化结果约为 $[1.07, -0.53, 1.60, 0]$ 。

这个例子里, LN 删除了原向量均值 1; RMSNorm 保留原向量的方向关系, 只控制整体 RMS。

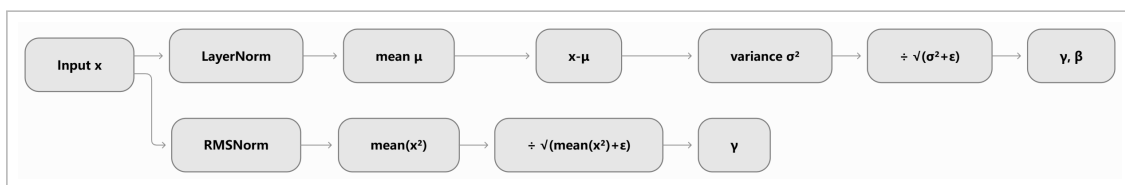


图 5 LayerNorm 与 RMSNorm 的计算步骤。

十、尺度性质

RMSNorm 最重要的数学性质是整体缩放不敏感。忽略 ϵ ，令 $x' = ax$ ，其中 $a > 0$ 。则

$$\text{RMS}(ax) = a \text{ RMS}(x),$$

所以

$$\frac{ax}{\text{RMS}(ax)} = \frac{x}{\text{RMS}(x)}.$$

假设 $x = [3, 4]$ 。RMS 是

$$\sqrt{\frac{9+16}{2}} = \sqrt{12.5}.$$

如果把输入整体扩大 10 倍，得到 $[30, 40]$ ，RMS 也会扩大 10 倍。两个向量经过 RMSNorm 后完全一致。网络因此对“上一层把所有通道一起放大多少倍”不敏感。

LayerNorm 还多一个整体平移不敏感。若 $x' = x + b\mathbf{1}$ ，减均值以后 b 会被抵消；RMSNorm 会保留这部分偏移。所以 RMSNorm 可以理解为：保留 re-scaling invariance，放弃 re-centering invariance。

RMSNorm 原论文的观点是，在很多深层网络中，尺度稳定承担了归一化的大部分优化作用，因此中心化步骤可以省略。

十一、几何解释

RMS 和二范数之间有一个很直接的关系：

$$\text{RMS}(x) = \frac{\|x\|_2}{\sqrt{d}}.$$

因此忽略 ϵ 时

$$\frac{x}{\text{RMS}(x)} = \sqrt{d} \frac{x}{\|x\|_2}.$$

也就是说，RMSNorm 保留向量方向，只把向量长度调整到固定量级。因为

$$\left\| \frac{x}{\text{RMS}(x)} \right\|_2^2 = d,$$

所以归一化后的二范数为 $\sqrt{d}$ 。

如果 $d = 4$ ，那么 RMSNorm 后、加入 γ 以前，向量的二范数约为 2。无论输入原来长度是 0.1、10 还是 100，只要方向一样，归一化以后都会落到相同尺度。

如果随后第 1 个维度的 $\gamma_1 = 2$ ，第 2 个维度的 $\gamma_2 = 0.5$ ，模型仍然可以学习“第 1 个通道更重要、第 2 个通道幅值更小”。所以 RMSNorm 控制的是整体尺度， γ 再恢复每个通道需要的相对尺度。

LayerNorm 的几何过程多一步：先从 x 中减去 $\mu\mathbf{1}$ ，把向量移动到零均值超平面，再进行尺度归一化。RMSNorm 直接沿当前方向调整长度。

十二、梯度

RMSNorm 的梯度也能说明它为什么对整体尺度不敏感。先忽略 γ ，记

$$r = \sqrt{\frac{1}{d}x^\top x + \epsilon}, \quad y = \frac{x}{r}.$$

先求 r 对第 j 个输入的导数。因为

$$r^2 = \frac{1}{d} \sum_k x_k^2 + \epsilon,$$

两边对 x_j 求导得到 $2r \partial r / \partial x_j = 2x_j / d$ ，所以

$$\frac{\partial r}{\partial x_j} = \frac{x_j}{dr}.$$

再对 $y_i = x_i / r$ 求导：

$$\frac{\partial y_i}{\partial x_j} = \frac{\delta_{ij}}{r} - \frac{x_i x_j}{dr^3}.$$

矩阵形式为

$$J = \frac{1}{r}I - \frac{1}{dr^3}xx^\top.$$

忽略 ϵ 后，把 x 代入 Jacobian 可以得到 $Jx = 0$ 。这个结果说明，沿着 x 自己的方向做一个很小的整体放大，对归一化结果的一阶影响为 0。它正好对应前面得到的缩放不变性。

可以把这个结论放回训练过程理解。若上一层权重暂时让一个 Token 的 hidden vector 整体放大 1%，RMSNorm 会把这部分整体尺度变化几乎消掉；真正能明显影响后续表示的是各维度之间相对比例发生变化。这样优化器更容易把精力放在表示方向和相对结构上。

十三、LayerNorm 与 RMSNorm

两者都对单个 Token 的 hidden 维度统计，也都与 batch size 无关。区别集中在中心化。

	LayerNorm	RMSNorm
统计范围	单 Token hidden 维	单 Token hidden 维
均值	计算	不计算
中心化	有	无
尺度统计	标准差	均方根
可学习 scale	γ	γ
可学习 bias	通常有 β	通常无
依赖 batch	无	无
训练/推理公式	一致	一致
缩放不变性	有	有
平移不变性	有	无

再看一个能清楚区分两者的例子。设

$$x = [1, 2, 3, 4], \quad x' = [101, 102, 103, 104].$$

两个向量只相差整体平移 100。LayerNorm 会把两者都减去各自均值，因此得到完全一样的归一化结果。RMSNorm 会看到 x' 的整体平方平均非常大，所以两个结果会不同。这个差异正好来自中心化是否存在。

现代 Transformer 中，Residual Stream 的分布通常已经受到初始化、残差结构、优化器和多层归一化共同约束，因此很多模型发现只保留尺度控制就足够稳定。RMSNorm 在这种环境下获得了很好的效果。

十四、数值实现

理论公式只有几行，工程实现还需要考虑低精度计算。大模型训练经常使用 FP16 或 BF16，如果直接在低精度下累计 $\sum_i x_i^2$ ，数值误差可能更明显，所以常见实现会先把输入转换到较高精度计算平方平均，再把结果转换回原数据类型。

RMSNorm 的典型实现可以概括为：先计算 $\text{mean}(x^2)$ ，再执行 reciprocal square root，最后计算 $x \cdot \text{rsqrt}(\text{mean}(x^2) + \epsilon)$ ，再乘可学习 weight。LayerNorm 还需要额外计算 mean、中心化和 bias。

现代 GPU 往往会把这些步骤融合到一个 kernel 中，因此单纯数 FLOPs 不能直接代表实际速度。RMSNorm 的优势更适理解解为：减少一次均值相关归约、一

次中心化和一个常见 bias 参数，同时保持训练需要的尺度稳定。归一化在几十层甚至上百层模型里反复调用，这种简化会持续累积。

十五、T5

T5 的论文是 *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer* (<https://www.jmlr.org/papers/v21/20-074.html>)。T5 使用的 LayerNorm 变体同样不减均值，也没有 additive bias，只根据平方平均缩放。

因此从计算形式上看，T5 的归一化已经非常接近 RMSNorm。它说明在大规模 Transformer 中，“只控制 hidden vector 的尺度”在 RMSNorm 广泛流行以前就已经被大型模型采用。

RMSNorm 论文从归一化理论和实验角度系统讨论了删除 re-centering 的合理性，T5 则从大规模序列模型实践上证明了类似结构的可行性。后来的 LLaMA 把 RMSNorm 明确写进 Decoder-only 大模型架构，进一步推动了这一设计成为常见选择。

十六、现代大模型

LLaMA 论文 *LLaMA: Open and Efficient Foundation Language Models* (<https://arxiv.org/abs/2302.13971>) 明确采用 Pre-Normalization，并使用 RMSNorm。典型 Decoder Block 可以写成

$$h = x + \text{Attention}(\text{RMSNorm}(x)),$$

然后

$$y = h + \text{FFN}(\text{RMSNorm}(h)).$$

Mistral 7B 论文: <https://arxiv.org/abs/2310.06825>。其公开实现同样使用 RMSNorm。Qwen2 Technical Report: <https://arxiv.org/abs/2407.10671>，公开实现中有 Qwen2RMSNorm。Gemma Technical Report: <https://arxiv.org/abs/2403.08295>，官方实现也采用 RMSNorm 及相关变体。

这些模型采用 RMSNorm 的原因可以连起来理解。第一，它沿单个 Token 的 hidden 维度统计，所以完全不依赖 batch。第二，它和 Pre-Norm 很容易组合，Residual Stream 保持直接通路。第三，它保留了对整体尺度变化的稳定性。第四，它的统计和参数比 LayerNorm 更简单。第五，现代 LLM 的归一化层会被高频调用，简化后的算子更容易做融合和高效实现。

假设一个 80 层 Decoder，每层 Attention 前和 FFN 前各调用一次归一化，那么一次前向至少要执行约 160 次归一化。单次 RMSNorm 比 LayerNorm 少掉的工作很小，但当它在每个 Token、每一层都重复出现时，简化就会不断累积。这也是“大模型基础算子为什么特别重视简单性”的直接原因。

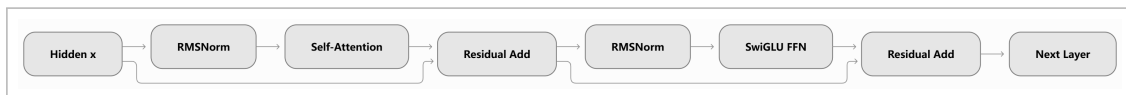


图 6 现代 Decoder Block 中常见的 Pre-RMSNorm 结构。

十七、发展脉络

从历史上看，归一化的发展主要沿三条线推进。

第一条线是统计范围。输入标准化处理整个数据集；BatchNorm 把统计放进网络内部，但依赖 batch；LayerNorm 把统计范围移到单个样本内部；InstanceNorm 和 GroupNorm 进一步说明，统计轴可以根据任务在通道、空间和分组之间调整。归一化方法之间很大一部分差异，都可以用“均值和方差到底在哪些维度计算”解释。

第二条线是归一化位置。原始 Transformer 使用 Post-LN，深层 Transformer 越来越多地采用 Pre-LN，让 Residual Stream 保持直接信息路径。归一化从“层输出的最后处理”移动到“子层计算之前”。

第三条线是统计量简化。LayerNorm 同时做 re-centering 和 re-scaling, RMSNorm 只保留 re-scaling。这个变化保留了 hidden vector 的尺度稳定和缩放不变性，同时减少统计和参数。

所以现代 LLM 中常见的结构可以写成

Pre-Norm + RMSNorm + Residual Connection.

Pre-Norm 决定归一化放在哪里，RMSNorm 决定怎样控制 hidden vector 的尺度，Residual Connection 提供直接的信息和梯度通路。三者共同组成了今天很多 Decoder Transformer 的基础结构。

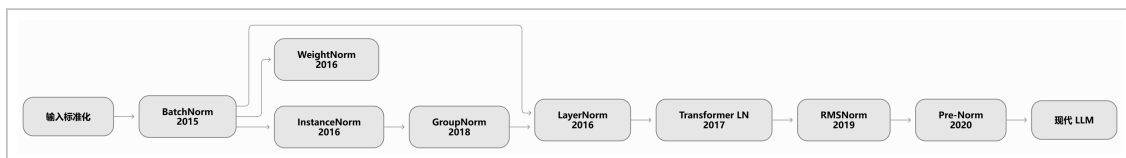


图 7 从 BatchNorm、LayerNorm 到 RMSNorm 的主要发展路线。

十八、对比

方法	统计对象	batch 依赖	典型场景
输入标准化	数据集特征	训练前统计	传统机器学习、网络输入
BatchNorm	同一特征跨 batch	有	CNN、大 batch 视觉训练
WeightNorm	权重长度与方向	无	参数重新参数化
LayerNorm	单样本 hidden 维	无	RNN、Transformer
InstanceNorm	单样本单通道空间	无	风格迁移、生成视觉任务
GroupNorm	单样本通道组与空间	无	小 batch 视觉模型
RMSNorm	单 Token hidden 维 RMS	无	现代 LLM、深层 Transformer

可以看到，发展过程越来越偏向“每个样本自己处理”。这和现代模型的训练、推理方式一致：batch size 可能变化，序列长度可能变化，线上推理甚至只有一个请求，但每个 Token 都始终拥有固定 hidden dimension，所以沿 hidden 维做归一化最稳定。

十九、公式总结

标准化的基本形式：

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}.$$

BatchNorm 和 LayerNorm 的公式很接近，主要差异在统计范围。BN 对同一特征沿 batch 统计；LN 对单个样本或 Token 沿 hidden 维统计。

LayerNorm：

$$\text{LN}(x)_i = \gamma_i \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_i.$$

RMSNorm：

$$r = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}, \quad \text{RMSNorm}(x)_i = \gamma_i \frac{x_i}{r}.$$

RMS 与方差的关系：

$$\text{RMS}^2(x) = \sigma^2 + \mu^2.$$

忽略 ϵ 时, RMSNorm 还满足

$$\text{RMSNorm}(ax) = \text{RMSNorm}(x), \quad a > 0.$$

现代 Decoder Transformer 中常见结构:

$$h = x + \text{Attention}(\text{RMSNorm}(x)),$$

$$y = h + \text{FFN}(\text{RMSNorm}(h)).$$

从 BatchNorm 到 RMSNorm, 最重要的变化可以概括为: 统计范围从 batch 转向单 Token, 归一化位置从 Post-Norm 转向 Pre-Norm, 统计量从“均值 + 方差”进一步简化为“均方根”。RMSNorm 最终保留了深层 Transformer 很需要的尺度控制, 同时让归一化算子更简单。

主要论文:

Ioffe, S., Szegedy, C. *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*. ICML 2015.

<https://proceedings.mlr.press/v37/ioffe15.html>

Salimans, T., Kingma, D. P. *Weight Normalization: A Simple Reparameterization to Accelerate Training of Deep Neural Networks*. 2016.

<https://arxiv.org/abs/1602.07868>

Ba, J. L., Kiros, J. R., Hinton, G. E. *Layer Normalization*. 2016.

<https://arxiv.org/abs/1607.06450>

Ulyanov, D., Vedaldi, A., Lempitsky, V. *Instance Normalization: The Missing Ingredient for Fast Stylization*. 2016.

<https://arxiv.org/abs/1607.08022>

Vaswani, A. et al. *Attention Is All You Need*. 2017.

<https://arxiv.org/abs/1706.03762>

Wu, Y., He, K. *Group Normalization*. ECCV 2018.

https://openaccess.thecvf.com/content_ECCV_2018/html/Yuxin_Wu_Group_Normalization_ECCV_2018_paper.html

Santurkar, S. et al. *How Does Batch Normalization Help Optimization?*. NeurIPS 2018.

<https://proceedings.neurips.cc/paper/2018/hash/905056c1ac1dad141560467e0a99e1cf-Abs.html>

Zhang, B., Sennrich, R. *Root Mean Square Layer Normalization*. NeurIPS 2019.

<https://proceedings.neurips.cc/paper/2019/hash/1e8a19426224ca89e83cef47f1e7f53b-Abs.html>

Xiong, R. et al. *On Layer Normalization in the Transformer Architecture*. ICML 2020.

<https://proceedings.mlr.press/v119/xiong20b.html>

Raffel, C. et al. *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. JMLR 2020.

<https://www.jmlr.org/papers/v21/20-074.html>

Touvron, H. et al. *LLaMA: Open and Efficient Foundation Language Models*. 2023.

<https://arxiv.org/abs/2302.13971>

Jiang, A. Q. et al. *Mistral 7B*. 2023.

<https://arxiv.org/abs/2310.06825>

Team Gemma. *Gemma: Open Models Based on Gemini Research and Technology*. 2024.

<https://arxiv.org/abs/2403.08295>

Yang, A. et al. *Qwen2 Technical Report*. 2024.

<https://arxiv.org/abs/2407.10671>